Multi-Objective Reinforcement Learning for Autonomous Driving and Adaptive Sensor Optimization

Zachary L. Johnson-Scott
University of Tennessee – Knoxville
ECE 517
Final Project – Milestone 2

Project Status:

What is your project idea?

This project investigates the feasibility of using multi-objective reinforcement learning to jointly optimize autonomous driving performance and adaptive sensor utilization, balancing safety, situational awareness, and energy efficiency.

What have you implemented so far?

A modular Gymnasium environment extends HighwayEnv with simulated RGB cameras, event cameras, radar, LiDAR, weather scenarios (Rain, Fog, Snow, Glare, Night), delayed measurements, grid observations, and energy accounting. The sensor (ON/OFF) and weather conditions effect how the simulator ground-truth is converted into information the RL agent can observe. Two notebooks demonstrate the environment and train an initial Expected SARSA agent.

What remains to be completed?

Calibrate environments sensor assumptions against literature, refine the driving/sensing objective reward system, and evaluate learned policies across more scenarios and training seeds. Independent sensor control /optimization, implement neural network approximation, and the proposal's nonterminating collision behavior remain future work.

Environment Design:

- **Environment:** What does your environment contain?

Component	Current implementation	Role
Highway	Three lanes, one controlled ego vehicle, twelve traffic vehicles	Simulates driving and traffic interactions
RGB cameras	Eight cameras facing N, NE, E, SE, S, SW, W, NW relative to the ego vehicle	Provide noisy object bearings without direct distance
Event cameras	Eight cameras facing the same directions as RGBs	Provide modeled bearing detections influenced by relative motion and illumination
Radar	One forward-facing sensor	Provides noisy range, bearing, and radial relative velocity

Multi-Objective Reinforcement Learning for Autonomous Driving and Adaptive Sensor Optimization

Component	Current implementation	Role
LiDAR	One sensor with 360° angular coverage	Provides noisy range and bearing, subject to range limits and occlusion
Weather and lighting	Clear, night, rain, fog, and glare presets	Modify detection probability, useful range, measurement noise, and false detections
Observation map	Ego-centered, 150 m radius; 2 m × 2 m cells	Represents available sensor evidence, uncertainty-related information, and measurement age
Timing	Physics at 100 Hz; agent decisions at 2 Hz	Updates traffic and sensor measurements between decisions
Configuration	Editable sensor, weather, grid, and reward parameters	Supports controlled experiments without rewriting the environment

- **State/Observation:** What information does the agent receive?

  Simulator truth is internal. The agent receives an ego-centered, 150-meter-radius map with 2-meter cells, represented by a 150 × 150 grid. Five layers describe occupancy evidence, measurement support, age, radial velocity, and velocity support. Additional observations provide the valid-cell mask, sensor activation/readiness/packet age, and ego speed and heading. Unknown space is distinguished from observed free space; the initial learning agent compresses observations into 110 features.

- **Action:** What actions can the agent perform?

  The environment currently accepts one of five driving actions left, right, maintain targets, faster, or slower plus an activation mask for the eighteen sensors. The first agent uses four predefined sensing modes (off, front, radar plus RGB, and all-on), giving twenty joint actions.

- **Reward:** How do you calculate rewards? (FOUND A FLAW IN FIRST RUNS)

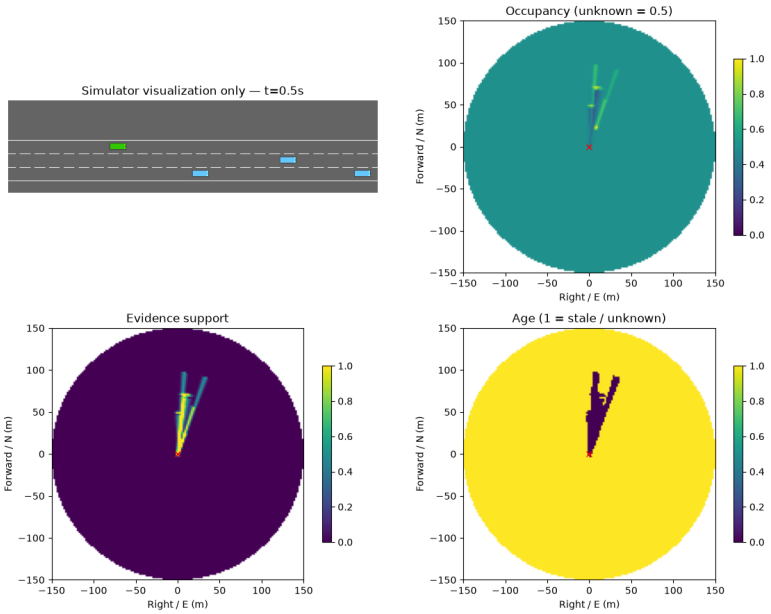
  Reward = driving reward from HighwayEnv – 0.001 x energy – 0.1 x uncertainty x decision duration. Uncertainty was added to account for sensor information.

  The driving component includes a speed reward up to +0.4 per decision and a −5 collision penalty. Energy integrates modeled power over simulated time; uncertainty is occupancy entropy within a local driving corridor. There is no explicit safe-time, distance, or completion bonus.

Multi-Objective Reinforcement Learning for Autonomous Driving and Adaptive Sensor Optimization

- **Transition:** How does the environment change after an action?

  Every decision advance 0.5 simulated seconds, using 100-Hz physics updates. Traffic moves, sensor activation changes, measurements are captured and delivered after warm-up/latency, old evidence ages, and the observation and reward are rebuilt. Collisions terminate an episode; reaching the duration limit truncates it.




```
# At severity=1: detection multiplier, useful-range multiplier, noise, clutter.
EFFECTS = {
    "clear": {},
    "night": {"rgb": (0.4, 0.6, 2.5, 1.5), "event": (0.65, 0.75, 2, 1.5)},
    "rain": {"rgb": (0.65, 0.7, 2, 2), "event": (0.65, 0.7, 2, 3),
             "radar": (0.9, 0.9, 1.3, 2), "lidar": (0.7, 0.7, 2, 3)},
    "fog": {"rgb": (0.35, 0.35, 3, 1.5), "event": (0.35, 0.35, 3, 1.5),
            "radar": (0.98, 0.98, 1.1, 1.1), "lidar": (0.4, 0.4, 3, 4)},
    "glare": {"rgb": (0.3, 0.6, 3, 2), "event": (0.75, 0.85, 1.5, 1.5),
              "lidar": (0.9, 0.9, 1.3, 1.3)},
}
```

Code helps model uncertainty caused by different weather patterns and sensor mated to those patterns. It contains, (detection probability, useful range, measurement noise, false detections rates); These affect what information the agent receive for observations. These made up by pure assumptions based on literature at the moment.

Environment Testing:

- How did you test your environment?

  Seventeen automated environment/agent tests pass, including Gymnasium interface checks, seeded reproducibility, sensor masking, timing, occlusion, energy accounting, coordinate

orientation, learning targets, and model save/load. Both experiment notebooks have executed successfully with saved plots and animations.

- What happens when the agent performs an action?

There are two actions spaces. First is the sensor activation which places them in several state spaces from on to off those models their potential dynamics. Activating certain sensor at given time step give the agent access to environment information. The second is the driving decisions which gives directional changes and speed targets.

- Does `reset()` correctly initialize the environment?

Initializes traffic and weather, clears pending measurements and energy, switches sensors off, and returns an unknown map. Repeating a configuration and seed reproduces the run.

- What technical challenges or constraints have you implemented?

Cameras supply noisy bearings without exact depth; radar adds range and radial velocity, while conventional LiDAR adds range. The implementation includes missed/false detections, simplified occlusion, stale measurements, and weather-dependent degradation; these are configurable abstractions, not validated physical sensor models



Next Steps:

- What will you implement next?
- What problems are you currently facing?

I will investigate the reward system and try to implement a neural network approximator. If you look up at the charts above you will notice adaptive sensing stopped crashing towards the final episodes while the Always On sensing started to regress. My assumption would be that always-on sensing should outperform adaptive when it comes to crashes as it has access to more information. However, what I found is the agent likely learned to drive slower when turning off sensors to save energy, as second order effect I believe the slower overall driving resulted in connection to not crashing. My reward system doesn't account for time to completion right now. This is small sample size but I believe that is a plausible assumption I will investigate.